A Unified Formalism for the Automated Verification of Arbitrary Program Logic

Paul Schmeing

March 2026

1 Abstract

The modern low level software development is plagued by an inherent gap between implementation and formal verification. Manual audits are tedious and economically expensive, while current security mechanisms focus on only one problem, like "Memory Safety". In this paper, we present **Otter**, a unified formalism for the automated verification of arbitrary program logic.

The core of the system is the **Otter Abstract Machine (OAM)**, which represents the program state in a directed graph consisting of domains. The abstraction of the physical hardware limits to provable, finite resource limits allows the creation of **Otter Proof Archives (OPA)**. This artifact ensures a cryptographic verified proof for the absence of runtime errors, undefined behavior, and logical invariant validations. With this, Otter transforms the audit-process from a reactive, human examination to a proactive mathematical certainty.

2 The Otter Abstract Machine (OAM)

The OAM is defined as a triple $\mathcal{M} = (G, \Sigma, \delta)$, where:

- G is the directed Property Graph representing all active domains and their relations.
- Σ is the Alphabet of Operations (Transitions).
- δ is the Transition Function $\delta : G \times \Sigma \to G \cup \{\bot\}$, where $\bot$ represents an unprovable (invalid) state.

For reasons of practicality, the OAM abstracts away specific hardware constraints, like memory and time, to focus on computational decidability and logical integrity. This means that every proof has to be executable in a finite amount of time and use a finite amount of memory, not considering whether the resulting hardware requirements are practical.

3 Domain Definition & Taxonomy

In the OAM, a domain $\mathcal{D}$ is considered the smallest unit of logical verification. We differentiate, between four fundamental classes of domains.

3.1 Primitive Domain ($\mathcal{D}_{atomic}$)

This domain is a standard domain as defined in the Set Theory. It describes a set of scalar values $\mathbb{S}$.

$$\mathcal{D}_{atomic} = \{v \mid v \in \mathbb{S} \land P(v)\} \tag{1}$$

where P is a predicate, for example an Interval $[0, 100]$ or disjunct values $\{2, 3, 5, 7\}$.

3.2 Ordered Domain ($\mathcal{D}_{ordered}$)

This type of domain is primarily used to describe complex data structures. The domain is defined as a tuple of n domains.

$$\mathcal{D}_{ordered} = (\mathcal{D}_1, \mathcal{D}_2, ..., \mathcal{D}_n) \tag{2}$$

Accessing a member m_i of an ordered domain $\mathcal{D}_{ordered}$ yields the sub-domain $\mathcal{D}_i$ while maintaining the constraints of the parent domain.

3.3 Proxy Domain ($\mathcal{D}_{proxy}$)

This type of domain is used to model references to another domain $\mathcal{D}_{target}$. A $\mathcal{D}_{proxy}$ is a singleton set containing the unique identifier ι of a target domain $\mathcal{D}_{target}$ within the OAM state graph.

$$\mathcal{D}_{proxy} = G_\iota \tag{3}$$

This domain is considered invalid when $\mathcal{D}_{target}$ successfully transitioned to $\mathcal{D}_\perp$.

3.4 Mapped Domain ($\mathcal{D}_{mapped}$)

This is arguably the most complex type of domain used in the OAM, it can be considered similar to a dependent type. It is used to define the domain depending on a different domain. It is a set of ordered domains containing D_{key} which describes the condition for this mapping and D_{value} which describes the domain.

$$\mathcal{D}_{mapped} = \{(\mathcal{D}_{key,i}, \mathcal{D}_{value,i}) \mid i \in I\} \tag{4}$$

A transition into a mapped domain is only valid if the current state s satisfies at least one $\mathcal{D}_{key,i}$, when more than one are satisfied only the intersection of the domains is accessible until further refinement. When more $\mathcal{D}_{key,i}$ are satisfied the resulting domain is defined as

$$\begin{aligned} I(x) &= \{i \in I \mid x \in \mathcal{D}_{key,i}\} \\ \mathcal{D}_{mapped} &= \bigcap_{i \in I(x)} \mathcal{D}_{value,i} \end{aligned} \tag{5}$$

3.5 Invalid Domain

A domain is considered invalid when the values it represents are no longer accessible or were destroyed. An invalid domain cannot be interacted with, if an access is tried this is considered to make this section unprovable and thus disproves the current check. An invalid domain is represented as $\mathcal{D}_{\perp}$, when this domain is found, the current proof is immediately considered false.

An empty domain ($\emptyset$) is not automatically considered invalid, but when a transition δ wants to access the domain $\mathcal{D}$ the transition immediately collapses to $\perp$.

Regarding $\mathcal{D}_{proxy}$ this means that when $\mathcal{D}_{target}$ is invalid, the proxy domain is automatically considered invalid, and neither can be used.

3.6 Structs

The OAM understands structs and considers them as an ordered domain ($\mathcal{D}_{ordered}$), the domain of a struct is a cartesian product of all domains it includes.

3.7 Capability

A capability describes an attribute that can be assigned to any node in G. Those capabilities can then be used as hints for checks. If a capability can be used by a check but is not explicitly required for this check, it is optimistic by assuming the non-existence mean this capability can be disregarded until disproven.

4 The Proof Graph

4.1 State transitions

A transition is the most basic operation, OAM can interpret and may change the state of G. The transitions are usable with almost any type of domain, but some exceptions exist

$$\mathcal{E} = \{\emptyset, \mathcal{D}_\perp\}$$
$$\forall \sigma \in \Sigma, \forall \mathcal{D} \in G : \delta(\mathcal{D}, \sigma) \notin \mathcal{D}_\perp \wedge \mathcal{D} \notin \mathcal{E}$$

A transition can generate an $\emptyset$ but no transition that interacts with a domain is allowed to accept an $\emptyset$, thus the domain has to be refined first. For example with a *switch* or *if*, this way a domain can be refined to be inhabited again.

4.2 Mapping logical memory to physical memory

Pointer arithmetic is for most systems a dead end, the OAM abstracts away the physical addresses and is only concerned about the intent the developer writes. This means that a member access of a struct is nothing different than a $\mathcal{D}_{proxy}$ pointing to the member of the struct. This allows the OAM to freely reorder and change alignment of fields without the logic breaking, when running the OAM corrects the pointer calculation by the difference between the logical intent and the physical layout in the memory.

$$ptr_{physical} = ptr_{logical} + \Delta mapping$$

4.2.1 Allowed Transitions

The OAM only supports a small set of transitions, from which more complex behavior emerges. In this section, we will only go over the transitions defined, not the emerging behavior.

Creating a Domain With the exception of the domain type $\mathcal{D}_{proxy}$ every newly created domain is added to the property graph G as a node. A $\mathcal{D}_{proxy}$ is created as an edge, pointing from a node or the stack to any other domain. When no target is provided or the targeted node is invalid, the $\mathcal{D}_{proxy}$ can still be created.

Deleting a Domain When a node is deleted, all $\mathcal{D}_{proxy}$ that point to this node are transformed to $\mathcal{D}_\perp$.

Arithmetic operations on a domain You can not perform any arithmetic operations on a $\mathcal{D}_{ordered}$ or $\mathcal{D}_{mapped}$.

For $\mathcal{D}_{proxy}$ any arithmetic operation is shifting the reference to a different point in G, for this operation it has to be verified that any new reference within G that is a valid outcome from the operation is a valid reference. A reference is valid, when the new G_ι is defined inside G and can be accessed at this point.

For $\mathcal{D}_{atomic}$, an operation modifies the scalar value of the domain.

Assigning a domain When assigning something to a domain, the domain type has to be kept unmodified. The old domain, if present, is discarded and the new domain is used from here on out. If discarding the old domain causes other domains to be affected those need to be updated before proceeding.

5 Formal Semantics

A program P is considered proven if for all inputs $E \in \text{Domain}(E)$ the path through the OAM is finite and no invariant is violated.

The time-complexity $T(n)$ is not considered in the OAM. A program with $T(n) = O(2^n)$ is considered as secure as an identical program with $T(n) = O(1)$, as long as both terminate and do not violate any constraints.

6 Otter Proof Archive (OPA)

6.1 Structure & Contents

6.2 Serialization

6.3 Signing